

Contracts should go into a White Paper - even at this late point

Abstract

This paper explains why, even at this stage and this late point, Contracts should go into a White Paper.

There are certain recent realizations and findings from existing practice in other languages that very strongly suggest this.

This paper explains those realizations and findings. The paper is abstract in the sense that it doesn't propose exactly how the things suggested in it should be achieved. While some might say "but we need a concrete proposal", such discussions are quickly lost in the weeds debating every work-in-progress detail. This paper is more about what we should be able to do, what our users should be able to do, and rough descriptions how to achieve those things.

The Problems of Contracts as designed in P2900

Contracts are rigid. Contracts are not flexible. Contracts are not customizable. Contracts are not extensible. We have feedback from real users that want to do things differently from P2900, and we're serving them poorly. Whenever they want to do something different, they need to go through all of WG21's processes to do something that amounts to a small (but significant) tweak.

Labels do not solve this problem. They continue to be core-language-heavy, and features-designed-by-committee-heavy. They don't seem to envision going into certain directions that some of our users want to go into, and it would take laborsome debate to convince a committee otherwise.

There are better ways to provide that flexibility, and there are existing practice examples of how to do it in Ada.

Various things that we've run into that should have been doable as third-party libraries, not language changes

I'm going to go over some use cases we have run into, some that we accommodated in P2900, some that we didn't, and how **all** of them should've been doable without changing a WG21 proposal, and should've been doable so that the users would have just written them themselves, deployed them at their leisure and at their chosen pace, and standardized later, as a separate optional exercise.

quick_enforce

The quick_enforce semantic, if it can even be called such, should've been doable as a third-party extension.

As hinted in my other work-in-progress paper, this would be doable with a couple of building blocks in hand:

1. a way to query the implementation what evaluation semantic configuration the user asked for
2. a way to raise a contract violation programmatically, which we're already considering in [P3290, Integrating Existing Assertions with Contracts](#).

Ahem. Those would be the building blocks needed to implement 'enforce', actually. Implementing quick_enforce needs just the first.

So what is actually required? Simply, this:

1. a predicate can query what the user configured the evaluation semantic to be, and if the user configured 'enforce', just abort()/__builtin_trap()/__fastfail() if the check yields false. Make the predicate noexcept so that it terminates if the check throws.
2. the user was already prepared for a contract violation to abort, the only difference here is that the violation handler wasn't called.
3. which isn't an outrageous thing to have, considering that with the labels proposal, you can do exactly the same thing, turn a user wish for 'enforce' into a 'quick_enforce', so it can't be a non-starter to do that programmatically, when our suggested roadmap provides this functionality anyway.

Note that I'm not talking about changing what a plain "x >= 0" in code means. I'm talking about how to express something like 'quick_enforce', functionally. You need a function for that, a function that queries the user config and acts accordingly. This is about expressing the desired functionality, at all, not just with the shortest syntax possible.

noexcept_enforce

The noexcept_enforce semantic, if it can even be called such, should've been doable as a third-party extension.

The building blocks are the same. Then just write a predicate that, if checks are enforced, runs the check, raises a violation if the check yields false, and terminates if the violation handler throws.

an 'enforce' that doesn't translate exceptions into contract violations

Contract predicates that do not translate exceptions into contract violations should've been doable as a third-part extension.

That translation has very different costs on ABIs that are not Itanium. So much so that the vendor of the biggest APIs like that considers the translation cost insurmountable and unacceptable.

And that, among other things, leads to a situation where our C++26 Contracts won't be portable. That's a very bad situation, and we should perhaps work harder to avoid it.

The same building blocks work. Evaluate the check if the user configured 'enforce', but don't evaluate the check in a noexcept envelope. If the check throws, just let it fly through.

How many hard-coded semantics do we actually (desperately) need?

Consider 'ignore'. It's implementable by a predicate that evaluates a check lazily, and doesn't evaluate it at all if the user config for the contract evaluation semantic was 'ignore'.

Again, that doesn't give you a plain "x >= 0", but we could still express the functionality. Wrap that condition into a function object, pass that function object to a predicate that checks the user config, and invokes the function object only if the user configured something other than 'ignore'.

Consider 'observe'. It's implementable by checking that the user wanted to turn checks on, evaluating a predicate, and then just...not terminating.

More on this in a bit, but first we need to look at a very particular aspect of Ada, and how it does 'observe'

A piece of existing practice

See [P3274](#), the 'expect' function template in Section 2, "Run-time checks". The author reports "everything that the expect() (real code on GitHub) can do is being used in production and will be wanted by some (probably many)."

The curious case of 'observe' in Ada

See [Ada Spark Assertion Pragmas](#).

In contrast to my earlier incorrect explanation on the reflectors, the Disable case isn't 'observe'. Both 'Ignore' and 'Disable' skip runtime evaluation of the condition at runtime.

But that doesn't actually mean Ada doesn't have 'observe'. It has all the building blocks you need to write it yourself.

Works like this: use a function as your predicate. If you want the function to just do 'observe', then

1. check the condition you wish to check
2. if the condition is false, call a log function (Ada doesn't have a standardized violation handler that operates as a side-effect of the predicate checks, like we do)
3. return true from the predicate.

That gives you 'observe'. The assertion may be Ignored/Disabled, and then it does nothing. And it can be Checked, and then it evaluates the condition, which then reports contract failures, but doesn't cause them to terminate, or in the case of Ada, to throw an assert exception.

Let's play with that idea a bit in the next section.

A guaranteed check is a programmatic building block, a check that may or may not be there is not

Functionality-wise, to implement every use case we have seen thus far, all we need is a way to query the user config, and then a check that is always evaluated.

So, such a check can do

- ignore, by just not running the sub-check at all
- observe, by running the sub-check, calling a violation handler, but not terminating
- enforce, by running the sub-check, calling a violation handler, and terminating if the handler returns normally
- quick_enforce, by running the sub-check, and then terminating
- a lighter-weight enforce, by running the sub-check, not catching exceptions possibly thrown by the check, and not turning those into terminate, calling the violation handler, and then terminating
- noexcept_enforce, by running the sub-check, calling the violation handler, and terminating if it returns normally or throws
- a combination of the above, controlled by some user configuration exposed by the compiler

In contrast, the flexible contract assertion of P2900 is not a programmatic building block. You can't build anything different on top of it. You can with labels, except if labels fail to provide some of the functionality described above. Labels could be good for computing an effective evaluation semantic, but will fall short if some semantic is not among the available possible options.

Labels

Labels look familiar. We've seen language<->library cooperation interfaces like that before. In coroutines, and coroutine traits. The language implements a particular algorithm, and provides hooks to customize some very well-thought aspects of it.

And then that doesn't work at all when library designers want to do something different, like async RAI, and we can't contort the existing library API to do what's needed, and we can't figure out how to even evolve the language part to use a new set of library hooks, if we had those.

And we end up regretting that.

A hard-coded algorithm that has customization hooks is very nice. Until you run into use cases that want to change the hard-coded parts. Like change whether exceptions are translated into contract violations.

Sure, we could add that as yet another thing that labels can customize, add more hooks.

If the committee has consensus for that. If not, then users just don't get what they need. Perhaps they should be allowed to *replace* such language-baked and compiler-baked algorithms completely?

Because that's what the different approach to exception->violation translation, namely an approach of not doing it, does. It goes against some of the SG21 principles, like the whole "ghost code" business.

But the thing is, for some users that's fine. Their APIs already throw, and throwing additional predicate exceptions is no big deal. Multiple other languages throw contract violations as exceptions, and their users deal with that without too much trouble.

And then there's the curious case of an always-checked assertion. Even better, an always-checked and always-terminating assertion.

Such assertions aren't "ghost code". They are normal code. It isn't completely outrageous any more that they would throw exceptions. They are always there. There's nothing conditional about them. It's quite unfortunate if they have to carry the exception->violation translation cost, when it would be altogether not at all surprising that they throw.

And why can't I do this customization by writing different functions, instead of writing some specialized class that models a particular concept and then implement various special hooks in it? Why does it need to be that complicated, and utterly novel? Why can't we have a simple customization possibility where I just write functions that do what I need, like I write for Every Other Need I have to express my domain-specific and application-specific needs? Why do I have to go through all the ceremony of writing a custom label?

So what can a White Paper tell us?

Or more specifically, what can more time spent tell us?

Funny question. It can tell us things that have been found out recently, just before this paper was written. Like how you can do 'observe' in Ada. And how, functionality-expressivity-wise, a guaranteed check is a building block for other things, which is a thing quite many of us dismissed as "definitely" not being the case.

And then it could give some of us a chance to formulate alternate, arguably better, designs further, to prototype them, to deploy them, and get us feedback on how such approaches work and how applicable users find them. And to deploy them side-by-side with the P2900 approach, and see which one fares better.

Rushing into an IS closes such alternative doors, at least to a fair extent. And it's **intended** to, as reported by some of the proponents of P2900.

And I disagree that we're ready to close those doors. During all of the time we have managed to spend on P2900, very few alternatives have actually been even prototyped, let alone deployed as prototypes. We have spent a whole lot of time discussing things in the abstract, and paper-designing The One True Alternative, and rejected other alternatives not by trying them out, but by discussing them.

That's literally what Design By Committee means.

And it ends up serving real practical users... poorly.